Course Name:	COMP 2233 Data Structures		
Lecturers:	Assoc. Prof. Dr. Ömer ÇETİN Research Assistant Giray KILIÇ Research Asistant Hatice ÇAĞLAYANLAR		
Lab. Number:	5	Lab. Date:	20.10.2026 / 23.10.2026
Objectives:	Pointers and singly linked lists in C: student registration, list operations, reversal, ordered merging, and safe dynamic memory management.		

Course Learning Outcomes (CLO):		Relevancy (1-5)
CLO (In1)	Students will be able to identify useful data structures to represent specific types of information.	5
CLO (Sk3)	Students will be able to design and implement linear data structures.	5
CLO (Co2)	Students will be able to design and implement efficient solutions for searching and sorting operations.	4

Pre-Lab Assignment

Review pointers, addresses (&), dereferencing (*), NULL, ->, and self-referential structures. CLO values are relevancy levels (1-5), not assessment marks.

Preparation for Both Experiments

1. Draw an empty list and a three-node singly linked list. Mark head, next, and NULL. Review malloc(sizeof(*node)), allocation checks, and free.

2. Explain when a function needs Student **head instead of Student *head. Compare array pointer arithmetic with following node->next.

3. Prepare pseudocode and a flowchart for the student menu (0-7) and ordered-list merging.

4. Manually merge [1, 3, 5] and [2, 3, 4, 6]. Expected result: [1, 2, 3, 3, 4, 5, 6]. Keep duplicates.

5. Plan tests for empty/one-node lists, duplicate numbers or values, missing students, head removal, reversal, and memory cleanup.

Development Environment

Use the attached C templates. main and scanf input are provided; complete the empty linked-list functions before compiling.

```
gcc -std=c11 -Wall -Wextra lab5_experiment1_template.c -o exp1
gcc -std=c11 -Wall -Wextra lab5_experiment2_template.c -o exp2
```

Self-Check

Why must next be saved before freeing a node? Why does current++ not traverse a linked list? Why can two sorted lists be merged in O(n + m)?

Laboratory Experiment

Experiment 1 (C): Student Registration System

1. Problem Description and Specifications

Implement a menu-driven singly linked list. Each Student node contains int number, char name[64], double grade, and struct Student *next. Start with head = NULL.

Student numbers must be positive and unique; names may contain spaces (maximum 63 characters); grades are in [0, 100]. Append new students. Updating changes name and grade, not the student number.

Menu: 1 Add; 2 Remove; 3 Update; 4 Search; 5 List; 6 Reverse; 7 Delete all; 0 Exit. Invalid conversions or record inputs report an error and terminate after cleanup; an invalid numeric menu choice shows the menu again.

2. Experimental Tasks

Complete lab5_experiment1_template.c without changing main or the function signatures:

find_student: traverse next links; return the matching node or NULL.

add_student: reject a duplicate number; allocate and initialize one node, check malloc, and append it. Leave the list unchanged on failure.

remove_student: reconnect the incoming link and free the selected node. Handle the head, middle, and last node; report a missing student.

update_student: find the record and replace name and grade; preserve its links. Report a missing student.

list_students: print number | name | grade (two decimals), or report an empty list.

reverse_students: reverse links in place using previous, current, and next; update head. Do not allocate or free nodes.

free_students: free the entire chain and set head to NULL. Support empty lists and repeated calls. Release all nodes on exit.

3. Tests and Expected Output

Add 101 / Ada Lovelace / 90 and 102 / Alan Turing / 80. Update 101 to Ada Updated / 95.5, search for 101, reverse, and list:

```
102 | Alan Turing | 80.00
101 | Ada Updated | 95.50
```

Also test duplicate registration, missing records, every removal position, empty/one-node reversal, delete-all then add-again, and input failure after allocation.

4. Complexity and Deliverables

For n students: search, append with duplicate checking, remove/update by number, list, reverse, and free take O(n) time. Reversal uses O(1) extra space; node storage is O(n).

Submit completed source, menu pseudocode/flowchart, test results, and complexity analysis. Avoid decorative output and source-code comments.

Experiment 2 (C): Merging Two Ordered Linked Lists

1. Problem Description and Specifications

Merge two singly linked lists of integers into one nondecreasing list. Inputs are already ordered; keep all values, including duplicates. Relink existing nodes instead of allocating a new result.

Each Node contains int value and struct Node *next. Empty lists and negative values are allowed. main stores the two heads in Node *lists[2] and reads nonnegative counts and values using scanf.

Decreasing input, invalid counts, or conversion failures report an error and terminate after freeing partial lists. Use integers representable by int.

2. Experimental Tasks

Complete lab5_experiment2_template.c:

append_node: allocate and append one node; return 1 on success. On allocation failure, report the error, leave the list unchanged, and return 0.

merge_lists: compare current nodes and link the smaller one to the result. On equality, select from the first list. When one list ends, attach the remaining chain.

Return the merged head and set both original heads to NULL. Do not allocate/free nodes during merging, copy values to arrays, or call a sorting routine.

print_list: print values on one line separated by spaces; report an empty list.

free_list: free all nodes and set the head to NULL. main frees the merged list on success and partial input lists on failure.

3. Example and Tests

```
List 1: 1 3 5
List 2: 2 3 4 6
Merged list: 1 2 3 3 4 5 6
```

Enter each list's element count, followed by its values. Test both empty, one empty, one node each, equal values, negative values, disjoint ranges, and interleaved values.

Test decreasing input and premature end of input after allocation. Verify that the merged list contains exactly n + m nodes and that cleanup does not free any node twice.

4. Complexity and Deliverables

Merging takes O(n + m) time and O(1) auxiliary space. Printing and freeing take O(n + m). Appending by traversal makes input-list construction O(n^2 + m^2); this is separate from merge complexity.

Submit completed source, merge pseudocode/flowchart, test results, and complexity analysis. Polynomial addition is an alternative topic, not required in this lab.

Additional Knowledge (Supplementary Reference)

Pointers and Dynamic Memory

&x obtains an address; *p accesses the pointed-to object. C passes arguments by value: pass &head to a Node ** parameter to change the caller's head. Use malloc/free, not C++ new/delete.

Check allocation before dereferencing; initialize every field. Save next before free. Each allocated node must be freed exactly once; never access freed memory.

Pointer Arithmetic and List Links

Within an array, p + 1 advances by sizeof(*p) bytes. Linked nodes are separate allocations: traverse with p = p->next, not p++. Empty head and the final next are NULL.

List Operations and Ownership

Insertion at the head is O(1). Searching and appending without a tail pointer are O(n). Updating a known node is O(1), but locating it by number is O(n).

To reverse: save next, redirect current->next to previous, and advance. The last processed node becomes head. This uses O(n) time and O(1) extra space.

During ordered merging, repeatedly take the smaller current node, then attach the remainder. Transfer ownership to the merged head and clear the original heads to prevent double-free.

Simple Input and Memory Checks

Check scanf's conversion count. %d expects int *; %lf expects double *. " %63[^\n]" reads a name with spaces into a 64-character buffer. Use names within the limit; extra characters remain unread.

Where supported, use -g -fsanitize=address,undefined when compiling completed sources. Test normal operation and cleanup on failure.

Attachments

A. Student template: lab5_experiment1_template.c
B. Student template: lab5_experiment2_template.c
C. Instructor solution: lab5_experiment1_ans.c
D. Instructor solution: lab5_experiment2_ans.c

Files accompany this document, not embedded. Templates provide main and empty list-function bodies; answer keys are separate.

References

1. Weiss, M. A. (1997). Data Structures and Algorithm Analysis in C (2nd ed.). Addison-Wesley.

2. Cormen, T. H., et al. (2022). Introduction to Algorithms (4th ed.). MIT Press.

3. Kernighan, B. W., & Ritchie, D. M. (1988). The C Programming Language (2nd ed.). Prentice Hall.

4. ISO/IEC 5807:1985. Documentation symbols and conventions for data, program and system flowcharts.